

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore  

Université Côte d’Azur, Inria, France

Enrico Tassi  

Université Côte d’Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the ELPI’s implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λ PROLOG (see [18, 19, 8, 13]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [14, 3, 4, 25, 9, 10]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [23, 24, 12], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [11] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [11]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [20] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:17

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !. % second one is cut away, as well as the choice
g D F :- f D E, f E F.    % points left by the call to r.

```

■ **Figure 1** Running example of a ELPI program

used by standard PROLOG abstract machines [26, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1 .

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [11]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceeding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The description of a program is given by the record $\mathbb{P}$ and is shared by both semantics. A program consists of a list of rules R together with a signature environment sigT . We delay the discussion of signatures to Section 6 where we describe the static analysis that concerns them.

```

Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record P := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.

```

■ **Figure 2** Definition of term, rule, program and substitution

A rule consists of a head term Tm and a list of premises. Each premise is an Atom , i.e. either the cut operator or a predicate call. Terms Tm include predicate symbols, data symbols, and variables. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λProlog to be based on the same mechanization.

The set of variables $\mathbf{V}$, predicates $\mathbf{P}$ and data symbols $\mathbf{S}$ are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the `finmap` library [17]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

Definition `bc` : $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \text{Tm} \rightarrow \Sigma \rightarrow \mathcal{F}_v * \text{list } (\Sigma * \text{list } \text{Atom})$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the `backchain` function takes a program, the set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2 , together with a substitution σ , as input, and returns an optional substitution σ' that extends σ . The expected behavior of `unify` is higher-order unification restricted to the pattern fragment, as explained in [18]. Therefore, if t_1 and t_2 unify under σ , then $\sigma' t_1 \equiv \sigma' t_2$, where σt is the notation for substitution application and $\equiv$ denotes the syntactic equality of its arguments.

2.1 The cut operator

There are quite a few variants of the cut operator. The one we are interested in, i.e. the one used in ELPI, is known as *hard cut* (see, e.g., the documentation of SWI-PROLOG [22]). Whenever the `backchain` function returns a non-empty list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by `backchain` for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in Figure 1 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$[(\sigma_1, [\text{r } \mathbf{A} \ \mathbf{B}, \ \mathbf{f} \ \mathbf{B} \ \mathbf{C}, \ !]), (\sigma_2, [\mathbf{f} \ \mathbf{D} \ \mathbf{E}, \ \mathbf{f} \ \mathbf{E} \ \mathbf{F}])]$

with $\sigma_1 := \{\mathbf{A} \mapsto 0, \mathbf{C} \mapsto \mathbf{R}\}$ and $\sigma_2 := \{\mathbf{D} \mapsto 0, \mathbf{F} \mapsto \mathbf{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1 , i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathbf{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathbf{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathbf{B} \mapsto 2\}$.

The next step of interpretation continues with `f B C` under σ_3 , that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4 , i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\mathbf{R} \mapsto 2\}$.

usiamo + ?

```

Inductive tree :=
  | KO | OK                                     (* failure and success *)
  | Todo of Atom                               (* unexplored branch *)
  | Or of option tree &  $\Sigma$  & tree             (*  $A \vee B_\sigma$  *)
  | And of tree & list Atom & tree.            (*  $A \wedge_{B_0} B$  *)

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree})$ 
Definition prune :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree}$ 
Inductive runT :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow \text{option } (\Sigma * \text{option tree}) \rightarrow \mathbb{B} \rightarrow \mathcal{F}_v \rightarrow \text{Prop}$ 

```

■ Figure 3 Signatures of the tree-based semantics

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for r and one from the second rule for g (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for g was selected. (In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

In Sections 3 and 4, we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\perp$ for **true** and **false**. The option instances are noted $\cdot t$ for *Some* t and $\square$ for *None*. Following the Mathematical Components library, on which we depend, we use $x.1$ and $x.2$ to denote the first and second projection applied to the pair x .

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The **Todo** node represents an unexplored branch (an **Atom**). When the atom is a **call**, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the **step** procedure described in Section 3.2.

The Rocq data type for And-Or trees is given in Figure 3. In addition to **Todo**, it provides two distinguished nodes, **KO** and **OK**, which represent respectively the smallest failed and successful tree.

The **Or** node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A . The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ , which becomes the current substitution when backtracking to B .

The **And** node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A . We call B_0 the *reset point* for B : it represents the continuation for all

```

Fixpoint next  $\sigma$   $t$  :  $\Sigma$  * tree :=
  match  $t$  with
  | Todo _ | KO | OK  $\Rightarrow$  ( $\sigma$ ,  $t$ )
  | Or  $\square$   $\sigma'$   $B \Rightarrow$  next  $\sigma'$   $B$ 
  | Or  $\cdot A$  _ _  $\Rightarrow$  next  $\sigma$   $A$ 
  | And  $A$  _  $B \Rightarrow$ 
    let ( $\sigma'$ ,  $pA$ ) := next  $\sigma$   $A$  in
    if  $pA$  == OK then next  $\sigma'$   $B$ 
    else ( $\sigma'$ ,  $pA$ )
  end.

Definition next_subst  $\sigma$   $t$  := (next  $\sigma$   $t$ ).1.
Definition next_tree  $t$  := (next  $\varepsilon$   $t$ ).2.

Definition success  $t$  := next_tree  $t$  == OK.
Definition failed  $t$  := next_tree  $t$  == KO.
Definition incomplete  $t$  :=
  if next_tree  $t$  is Todo _ then  $\top$  else  $\perp$ .

```

■ Figure 4 next and derived functions

alternatives of A except the first. That is, when backtracking occurs within A , the subtree B is replaced by the atoms in B_0 . An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 5b. For compactness, we depict And and Or nodes as n -ary (rather than binary), with children associating to the right. The KO node acts as the terminating element of an Or list, while OK is the terminating element of an And list.

3.1 The tree-based semantics

The tree is constructed incrementally by two recursive functions, **step** and **prune**. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to the **next_tree** (in Figure 4). In the snippet we also identify special kinds of trees: depending on the **next_tree**, we distinguish between *success*, *failed* and *incomplete* trees. Since all functions must terminate in Rocq, we define their transitive closure as the inductive relation **runT**.

Intuitively, **runT** iterates calls to **step** on incomplete trees to evaluate the **Todo** node. Upon failed trees, it calls **prune** to find the next alternative and continues from there, if one exists. We detail **step**, **prune**, and **runT** in Sections 3.2–3.4.

In Figure 5 we mark with a black arrow the result of **next_tree**.

3.2 The step procedure

The **step** procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a **tag**, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

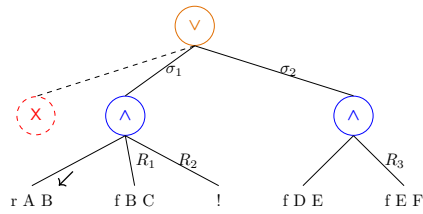
The **tag** (see Figure 3) indicates which kind of step was performed. **Failure** and **Success** are returned for trees that satisfy, respectively, the **failed** and **success** tests. **CutBrothers** indicates the interpretation of a superficial cut: **step** was called on an And-layer and its **next_tree** is the cut atom. Finally, **Expanded** accounts for the interpretation of predicate calls and non-superficial cuts.

The **step** procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

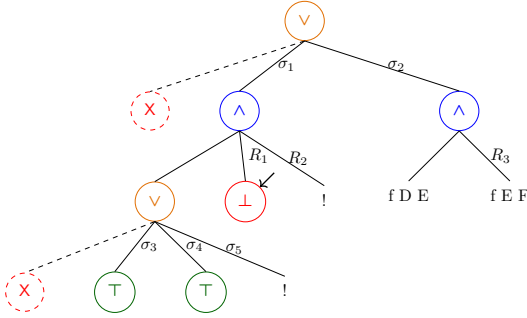
¹ The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2 , and R_3 are reset points and correspond respectively to the lists of atoms $[f \ Y \ Z, !]$, $[!]$, and $[f \ Y \ Z]$.

g 0 R

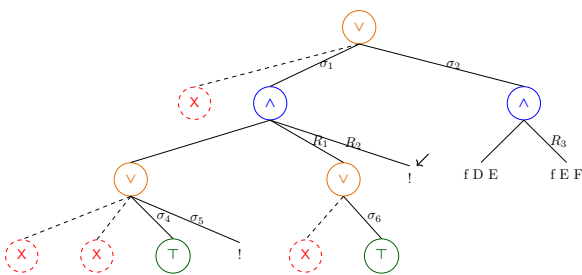
(a) Initial query



(b) Tree after step on g 0 R

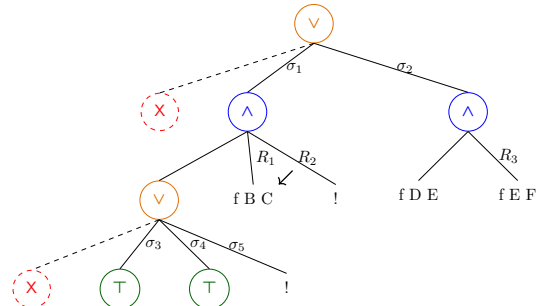


(d.1) step on Figure 5c

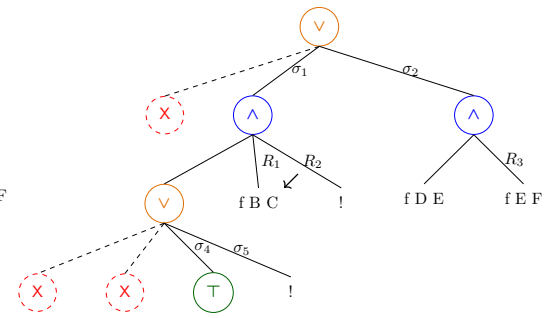


(e) step on Figure 5d.2

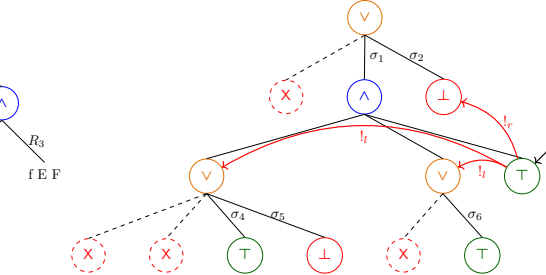
■ Figure 5 Some tree representations¹



(c) step on Figure 5b



(d.2) prune on Figure 5d.1



(f) step on Figure 5e

180 ► **Lemma 1** (*succ_step_iff*). $\forall p \ v \ \sigma \ t, \text{success } t \leftrightarrow \text{step } p \ v \ \sigma \ t = (v, \text{Success}, t).$

181 ► **Lemma 2** (*fail_step_iff*). $\forall p \ v \ \sigma \ t, \text{failed } t \leftrightarrow \text{step } p \ v \ \sigma \ t = (v, \text{Failed}, t).$

182 **step** expands **Todo** nodes by calling **backchain**, which returns a list l of alternatives. If l
 183 is empty, then the current call is replaced by **K0**. Otherwise, it is replaced by a right-skewed
 184 tree of **Or** nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf
 185 is **OK**; otherwise, it is a right-skewed tree of **And** nodes.

186 Figure 5b depicts the tree corresponding to the running example (Section 2.1) as it
 187 undergoes calls to **step**. We run query $q \ 0 \ R$ against the program P in Figure 1. Let c_0 ,
 188 c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2
 189 correspond respectively to the premises of rules r_1 and r_2 in P . We need the initial $\square$ subtree
 190 so that we can attach a substitution to c_1 . In particular, c_1 (resp. c_2) is associated with
 191 substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are
 192 marked with the R symbol: $R_1 := [f \ Y \ Z, !]$, $R_2 := [!]$, and $R_3 := f \ Y \ Z$. Intuitively, they
 193 record the lists of atoms used to generate the corresponding subtree. Other examples of
 194 **step** triggering backchaining appear in Figures 5c–5e.

sottolineare le
sostituzioni

195 3.2.1 The interpretation of a cut

196 The step corresponding to a cut performs two actions: (i) the cut node is replaced by **OK**; and
 197 (ii) the subtrees in the scope of **Cut** are pruned. More precisely, (ii) prunes the left siblings
 198 of the **Cut** node (in the same **And**-layer, denoted $!_l$) and prunes the right uncles of **Cut** (in
 199 the one-level-up **Or**-layer, denoted $!_r$).

200 An example of the impact of cut is shown in Figure 5f. The red arrows indicate the scope
 201 of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to
 202 each pointed subtree. We note that the evaluation of a cut keeps the path leading to the
 203 cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under
 204 substitution σ_5 (which also succeeds) is replaced by **K0**. This amounts to discarding the third
 205 rule of predicate r (see Section 2.1).

206 3.3 The prune procedure

207 When **backchain** returns an empty list, **step** produces a *failed* tree and the computation
 208 must backtrack. The **prune** procedure is responsible for producing the new tree from which
 209 the computation can continue.

210 In Figure 5d.1 we encounter a failure because no rule applies to the atom $f \ B \ C$ under
 211 substitution σ_3 , which maps B to 0. The **prune** procedure prunes the path leading to this
 212 failure and resets the current sub-query to its original shape. More precisely, it kills the **OK**
 213 node under σ_3 (since that branch has been fully explored and produced no solution), and
 214 then replaces the **K0** node with the information stored in the reset point R_1 . This restores
 215 the call $f \ B \ C$, which can then be reconsidered under substitution σ_4 .

216 Furthermore, **prune** serves another important purpose. Its behavior depends on the
 217 boolean flag it receives as input: **prune** $\perp A$ recursively removes all failures (if any) from
 218 A , whereas **prune** $\top A$ removes the first success in A . For example, the tree in Figure 5f
 219 contains a successful path that maps R to 2. Calling **prune** on this tree returns $\square$, since there
 220 are no alternatives in the tree after the success.

221 Some interesting properties of **prune** are shown below; they illustrate how **prune** comple-
 222 ments **step** with respect to failed, success, and **incomplete** trees.

223 ► **Lemma 3** (*incomplete_prune_id*). $\forall b \ t, \text{incomplete } t \rightarrow \text{prune } b \ t = .t.$

224 ► **Lemma 4** (`prune_Some`). $\forall b \ t \ t', \text{prune } b \ t = \cdot t' \rightarrow \text{failed } t' = \perp.$

225 ► **Lemma 5** (`prune_None`). $\forall t, \text{prune } \perp \ t = \square \rightarrow \text{failed } t.$

226 3.4 The `runT` inductive

227 The inductive relation `runT` relates four inputs to three outputs. The inputs are a program,
228 a set of variables, an initial substitution σ , and a tree t . The outputs are (i) an optional
229 result r of the form $\cdot(\sigma', t')$, (ii) a boolean b , and (iii) an updated set of variable names.

230 The main output is r : if $r = \square$, interpretation fails; otherwise σ' is the computed solution
231 substitution and t' represents the remaining, not-yet-explored alternatives. The boolean b
232 records whether a superficial cut was evaluated during execution, and the updated variable
233 set tracks freshly generated names.

234 `runT` has four cases, depicted as inference rules in Figure 6. (*StopT*) If `next_tree` t is a
235 success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned
236 of its successful path. (*StepT*) If `next_tree` t is an atom, then `step` is invoked to evaluate t , and
237 `runT` is called recursively on the updated tree. (*BackT*) If `next_tree` t is a failure, then `prune`
238 is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it.
239 Finally, (*FailT*) accounts for trees with no solutions.

240 These rules are deterministic:

241 ► **Lemma 6** (`runT_det`). $\forall p \ v_0 \ \sigma \ t \ r \ r' \ b \ b' \ v \ v', \text{runT } p \ v_0 \ \sigma \ t \ r \ b \ v \rightarrow$
242 $\text{runT } p \ v_0 \ \sigma \ t \ r' \ b' \ v' \rightarrow r' = r \wedge v' = v \wedge b' = b.$

242 The proof is by induction on the first `runT` derivation and is immediate, because the rules are
243 selected based on `next_tree` t . In particular, the hypotheses `success` t , `incomplete` t , and `failed`
244 t are mutually exclusive. Moreover, by ??, the hypothesis of *FailT* implies *failed* t ; hence *FailT*
245 is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different
246 requirements on the output of `prune`.

$$\begin{array}{c}
 \frac{\text{success } t \quad \text{next_subst } \sigma \ t = \sigma' \quad \text{prune } \top \ t = t'}{\text{runT } p \ v \ \sigma \ t \cdot(\sigma', t') \perp v} \text{StopT} \\
 \\
 \frac{\text{incomplete } t \quad b' = (\text{tg} = \text{CutBrothers}) \vee b \quad \text{step } p \ v \ \sigma \ t = (v', \text{tg}, t') \quad \text{runT } p \ v' \ \sigma \ t' \ r \ b \ v''}{\text{runT } p \ v \ \sigma \ t \ r \ b' \ v''} \text{StepT} \\
 \\
 \frac{\text{failed } t \quad \text{prune } \perp \ t = \cdot t' \quad \text{runT } p \ v \ \sigma \ t' \ r \ n \ v'}{\text{runT } p \ v \ \sigma \ t \ r \ n \ v'} \text{BackT} \\
 \\
 \frac{\text{prune } \perp \ t = \square}{\text{runT } p \ v \ \sigma \ t \ \square \perp v} \text{FailT}
 \end{array}$$

■ **Figure 6** Rule system for `runT`

247 3.5 Properties of `runT`

248 Working with `runT` allows to prove some interesting properties about the behavior of the cut operator
249 wrt disjunction, i.e. the `Or` node. We have proven several properties that are available at [this link](#).
250 Below we show two of these lemmas.

251 ► **Lemma 7** (`runSST_or`). $\forall p \ v \ v' \ \sigma \ \sigma' \ l \ l' \ \sigma_1 \ r, \text{runT } p \ v \ \sigma \ l \cdot(\sigma', \cdot l') \top v' \rightarrow$

Definition `save_gs a g b :=`
`[seq (x, a) | x <- b] ++ g.`

Definition `save_as a g b :=`
`[seq (x.1, save_gs a g x.2) | x <- b].`

Definition `stepS p v t σ a g :=`
`let (v', r) := backchain p v t σ in`
`(v', save_as a g r).`

$$\begin{array}{c}
 \frac{}{\text{runS } p \ v \ ((\sigma, \emptyset) :: a) \cdot (\sigma, a)} \text{StopS} \qquad \frac{\text{runS } p \ v \ ((\sigma, g) :: ca) \ r}{\text{runS } p \ v \ ((\sigma, (cut, ca) :: g) :: _) \ r} \text{CutS} \\
 \\
 \frac{\text{stepS } p \ v \ t \ \sigma \ a \ g = (v_1, a') \quad \text{runS } p \ v_1 \ (a' ++ a) \ r}{\text{runS } p \ v \ ((\sigma, (call \ t, _) :: g) :: a) \ r} \text{CallS} \quad \frac{}{\text{runS } p \ _ \ \emptyset \ \square} \text{FailS}
 \end{array}$$

■ **Figure 7** Rule system for `runS`

$\text{runT } p \ v \ \sigma \ (\cdot l \vee r_{\sigma_1}) \cdot (\sigma', \cdot (\cdot l' \vee KO_{\sigma_1})) \perp v'.$

252 ▶ **Lemma 8** (`runNT_or`). $\forall \ p \ v \ v' \ \sigma \ t \ \sigma_1 \ t', \text{runT } p \ v \ \sigma \ t \ \square \top v' \rightarrow$

$\text{runT } p \ v \ \sigma \ (\cdot t \vee t'_{\sigma_1}) \ \square \perp v'.$

253 ▶ **Lemma 9** (`run_orSST`). $\forall \ p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ l \ l' \ r \ r', \text{runT } p \ v \ \sigma \ (\cdot l \vee r_{\sigma_1}) \cdot (\sigma', \cdot (\cdot l' \vee r'_{\sigma_1})) \perp v' \rightarrow$
 $\exists \ b, \text{runT } p \ v \ \sigma \ l \cdot (\sigma', \cdot l') \ b \ v' \wedge r' = \text{if } b \text{ then } KO \text{ else } r.$

254 In Lemma 7, we evaluate the tree l . This evaluation produces a success with signature σ' and a
 255 new tree l' . During the evaluation, a superficial cut in l is traversed. This implies that the evaluation
 256 of $\cdot l \vee_{\sigma_1} r$ returns σ' as substitution, and the alternatives are $\cdot l' \vee_{\sigma_1} KO$. The right-hand side of the
 257 `Or` node is replaced with `KO` by $!_r$.

258 In Lemma 8, we are in a similar scenario: t evaluates a superficial cut but, this time, produces a
 259 failure. This implies that the evaluation of $\cdot t \vee_{\sigma_1} t'$ also fails.

260 Finally, in Lemma 9, we evaluate $\cdot l \vee_{\sigma_1} r$. The evaluation succeeds, producing the result
 261 $\cdot (s_1, \cdot l' \vee_{\sigma_1} r')$. This means that l still has l' as unexplored alternatives. We deduce that the
 262 evaluation of l produces the substitution σ_1 with alternatives l' . However, we have no information
 263 about whether a superficial cut has been crossed in l . If this is the case, we know that r' is cut away
 264 by $!_r$; otherwise, r' has not been explored and is equal to r .

265 From these lemmas, it is clear how the order in which alternatives are evaluated changes the
 266 behavior of the program.

4 Stack semantics

268 We now introduce the stack semantics. The interpreter is close to that of the ELPI language and
 269 provides an operational semantics that is similar to the one proposed by Pusch in [20], which is
 270 in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch
 271 mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present
 272 in the Warren Abstract Machine [26, 1].

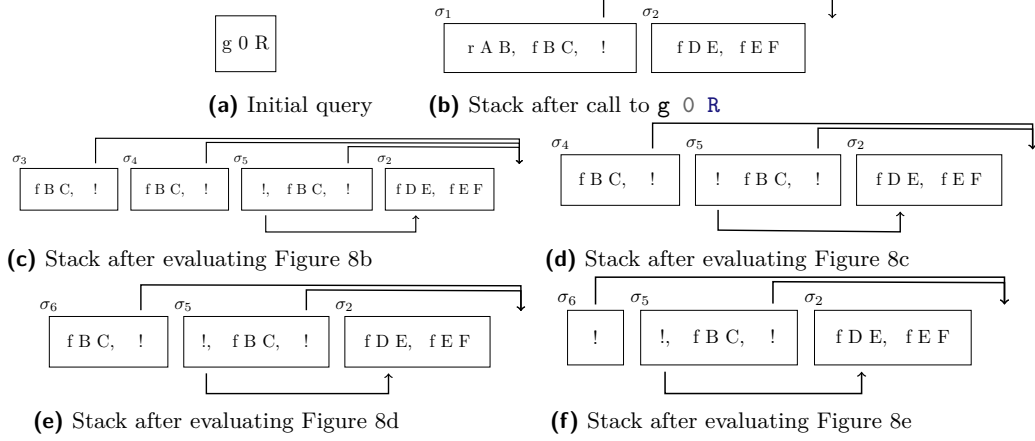
273 We represent a state of the ELPI language thanks to the two mutual inductives shown below.

Inductive `alts := nilA | consA of (Σ * goals) & alts`
with `goals := nilG | consG of (Atom * alts) & goals .`

274 A stack is an enhanced two-dimensional list, called *alts* in the inductive. The outermost list
 275 represents a list of alternatives in disjunction, each accompanied by the substitution that should
 276 be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in
 277 conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep
 278 track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to*
 279 alternatives.

280 The stack interpreter is called `runS` and has the following type.

se riesco success
 (A or B) = suc-
 cess (B or A) if
 no sup cut in A
 and B



■ **Figure 8** Example of `runS` execution

281 **Inductive** `runS` ($p: \mathbb{P}$) : $\mathcal{F}_v \rightarrow \text{alts} \rightarrow \text{option } (\Sigma * \text{alts}) \rightarrow \text{Prop}$ (1)

282 The inductive `runS` represents a routine taking as input a set of variables and a list of alternatives
 283 and returning an option. $\square$ stands for interpretation with no solution, whereas $\cdot(\sigma, a)$ represent a
 284 successful interpretation with σ as solution and a as the list of non-yet-explored choice points.

285 The rule system for `runS` is given in Figure 7 and is made by 4 cases. We distinguish two main
 286 cases: if the list of alternatives is empty, (*FailS*) we have a failure: we stop the computation and
 287 return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter evaluates the first choice
 288 point h under the substitution σ_0 . If h is empty, (*StopS*), then we have a success, i.e. we return
 289 the input σ_0 and leave a as the future choice points. Otherwise, h is the list $(t, ca) :: g$ where t is a
 290 the first atom to evaluate, ca is its cut-alternative list and g is the list of future conjuncts. If t is a
 291 cut (*CutS*), we need to keep evaluating g under σ_0 but we change the alternatives to ca . Finally, in
 292 the case of a call (*CallS*), we backchain the call in the program. If rs is the list of rule premises
 293 returned by `backchain`, then each list of premises is mapped to a list of goals (`goals`) have a as
 294 cut-alternative and the global list is translated to a list of alternative (`alts`).

295 *Example of execution* We propose an example of the execution of `runS` in Figure 8. The arrows
 296 in the images represent the cut-alternative pointers of the cut. We have not drawn arrows coming
 297 out of atom-calls since the interpreter ignores the cut-alternatives in this case (cf. *CallS*). The
 298 evolution of the stack in Figure 8 mirrors the example in Section 2.1: we evaluate the first goal
 299 of the first alternative. During backchaining, we add a pointer to the remaining alternatives to
 300 each new cut operator. If a cut is evaluated, we replace the remaining alternatives with the cut-to
 301 alternatives. For example, in Figure 8f, evaluating the cut discards the second and third elements in
 302 the stack. Backtracking is performed by simply removing the first alternative from the stack, as
 303 shown in Figure 8d.

304 *Remark* We want finally to point out how the lemma that we were able to prove in Section 3.5
 305 are not easy to be expressed in the stack semantics. This is because there is no sufficient structure in
 306 the stack to understand what is the scope of the cut operator. For example, in a stack made of these
 307 alternatives $a_0, a_1 \dots a_n$, we have no clear idea of what happens if the first alternative a_0 succeeds:
 308 there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives
 309 $a_1 \dots a_n$? More than that, it is also possible that the state is hill-formed and the cut does not point
 310 to a suffix in the list of alternatives.

5 Equivalence of the two semantics

312 In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree
 313 and the stack of alternatives. In particular we define a translation from trees to stacks, called `ttl`.
 314 We do not explicitly provide the converse translation, an economy allowed by the proof technique we

```

315 Fixpoint valid_tree A :=
316   match A with
317   | Todo _ | OK | KO  $\Rightarrow$  T
318   | Or  $\square$  _ B  $\Rightarrow$  valid_tree B
319   | Or  $\cdot$ A _ B  $\Rightarrow$  valid_tree A &&
320     ((B == KO) || B.base_or B)
321   | And A BO B  $\Rightarrow$  valid_tree A &&
322     if success A then valid_tree B
323     else B == big_and BO
324   end.
325
326 Fixpoint t2l t  $\sigma$  (cp: alts) : alts :=
327   match t with
328   | OK  $\Rightarrow$  [( $\sigma$ ,  $\emptyset$ )]
329   | KO  $\Rightarrow$   $\emptyset$ 
330   | TA a  $\Rightarrow$  [( $\sigma$ , [(a,  $\emptyset$ )])]
331   ...

```

(a) Valid tree

(b) Tree to list

■ **Figure 9** Valid tree and Tree to list

315 employ, inspired by [2].

316 Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion
 317 of *valid tree*. The **tree** datatype indeed contains trees that cannot be generated by **runT** via **step** or
 318 **prune**, for example all the trees that do not correspond to a depth-first left-to-right tree construction
 319 strategy.

320 5.1 Valid trees (valid_tree)

321 The class of valid trees is characterized by the function shown in Figure 9a.

322 While all base cases of tree are considered valid, we need to analyze carefully the cases for the
 323 **Or** and **And** constructors.

324 For the **Or** constructor, we distinguish two cases depending on whether the left hand side is
 325 present. If it is not present, then the right hand side must be a valid tree. Otherwise, the left
 326 hand side must be a valid tree and the right hand side must either be the **KO** tree or be unexplored.
 327 The first case occurs when the right hand side is cut out by the evaluation of a superficial cut
 328 present in the left hand side. In the latter case we use the **base_or** predicate to check that **B** is
 329 the right-skewed tree formed by a disjunction of conjunctions that is generated after a succesful
 330 backchain to a **call**.

331 For the **And** constructor, the left hand side is required to be a valid tree. The shape of the
 332 right hand side depends on whether the left hand side is a success. If it is not successful, then the
 333 right hand side must not be explored, i.e. **B** must be the right-skewed tree containing conjunctions
 334 of premises atoms. This is enforced using the **big_and** produce that generates a tree out of the
 335 reset point B_0 , the same procude used to transform each alternative output by **backchain** into the
 336 And-layer of the resulting tree. When the left hand side is successfule, then the right hand side may
 337 have been explored, and consequently must be a valid tree.

338 5.2 From trees to stacks (t2l)

339 The **t2l** recursive function translates a tree to a stack. For space constraints Figure 9b only gives
 340 the base cases, while we describe the recursive cases in the following text.

341 The function **t2l** takes as input the tree t_0 , a substitution σ_0 , and a list of choice points cp .
 342 These choice points represent alternatives that appear at the same level of the current tree, such as,
 343 for example, the right siblings of an **Or** node. The substitution is used to determine under which
 344 substitution the subtree being compiled lives.

345 The **OK** node represent one succesful alternative, thereofre it is translated into a singleton list
 346 with the empty list of goals. The **KO** node represent a failure, i.e. it is mapped to the empty list of
 347 alternatives. Finally, atoms are mapped to one alternative containing a single goal with the empty
 348 cut-to alternative list. They cannot point to cp since they are not right-uncle subtrees, i.e. cp are
 349 alternatives that will be actually discarded if the atom is a cut.

DAVVERO?

superficial?

350 The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B . The
 351 alternatives of B are valid choice points for A and should be passed to the translation of A . The two
 352 list of alternatives can be concatenated and, each atom, should add cp as new cut-to alternatives: A
 353 and B are one level lower wrt cp and therefore the cut they have should point to cp .

354 The translation of $A \wedge_{B_0} B$ starts with the translation of A . If the translation of A is an
 355 empty list we return the empty list of alternatives without even touching B nor B_0 , since an empty
 356 translation for A indicates failure and this in turn implies the failure of the entire conjunction. When
 357 the translation of A is a list $(\sigma_0, g) :: a$ the B node represent the continuation of the first alternative
 358 g , whereas B_0 is the continuation of each alternative in a .

359 5.2.1 Example of t2l

360 The translation of the trees in Figure 5 is given in Figure 8. The letters in the figures relate the
 361 trees to the corresponding letters in the stack figures. For example, the tree 5b is translated into
 362 8b. We note that the two trees 5d.1 and 5d.2 are both translated into 8d, showing that $t2l$ is not
 363 injective; that is, there are different trees that map to the same stack. This means that there are
 364 transitions in **runT** that are not visible in **runS**.

365 As a more precise example of the behavior of **t2l**, we consider the tree 5c, and in particular its
 366 deepest **Or** node. This subtree shows a disjunction with four children. The first is ignored, since it is
 367 $\square$. The success node below σ_3 has **f B C** and the cut operator as continuation, corresponding to the
 368 first cell in the stack in 8c. We note that the cut operator points outside the stack, since the scope
 369 of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is
 370 the list of goals **f B C, !**; this is translated into the second cell of the stack in 8c. Finally, the cut
 371 operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative in Figure 8c.
 372 We can see that the first cut points to the translation of the subtree under σ_2 , since it is one **Or**-layer
 373 above its right uncles.

374 5.3 Equivalence proof

375 The tree-based semantics exposes more information so to be usable to express the lemmas in
 376 Section 3.5. For the equivalence this information is irrelevant, hence define a version of **runT** that
 377 hides it.

378 ► **Definition 10** ($\text{runT}'(p \ v \ \sigma \ t \ r)$). $\exists \ b \ v', \text{runT } p \ v \ \sigma \ t \ r \ b \ v'$.

379 From Figures 5 and 8, we observe that, upon backtracking, **runT** may perform more reduction
 380 steps than **runS** before returning a solution or a failure. These transitions are called silent transitions
 381 and must be skipped when proving the equivalence between the two semantics. Therefore, we state
 382 the following lemma.

383 ► **Lemma 11** (pruneF_t2l). $\forall \ t \ t' \ b, \text{valid_tree } t \rightarrow \text{failed } t \rightarrow \text{prune } b \ t = \cdot t' \rightarrow$
 $\forall \ l \ \sigma, \text{t2l } t \ \sigma \ l = \text{t2l } t' \ \sigma \ l$.

dire di v

384 **Proof.** By induction on the tree t . ◀

385 The first direction of the equivalence states that the execution of a valid tree is matched by
 386 the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives
 387 returned as a tree correspond to those returned as a stack.

388 ► **Theorem 12** (tree_to_elpi). $\forall \ p \ t \ \sigma \ r, \text{let } v := \text{vars_tree } t \ \backslash \ \text{vars_sigma } \sigma \text{ in}$
 $\text{valid_tree } t \rightarrow$
 $\text{runT}' \ p \ v \ \sigma \ t \ r \rightarrow$
 $\text{let } a := \text{t2l } t \ \sigma \ \emptyset \text{ in}$
 $\text{let } r' := \text{if } r \text{ is } \cdot(\sigma', t) \text{ then } \cdot(\sigma', \text{t2l } (\text{odflt } KO \ t) \ \sigma \ \emptyset) \text{ else } \square \text{ in}$
 $\text{runS } p \ v \ a \ r'.$

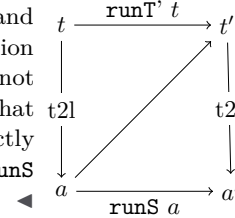
Proof. We proceed by induction on the execution of `runT'`. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: `step` evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of `t2l`; however, thanks to Lemma 11 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if `prune` returns $\square$ when trying to erase failure in t , then `t2l` applied to t returns the empty list. $\blacktriangleleft$

The converse direction is stated following [2]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns an unexplored tree that translates to the unexplored stack.

► **Theorem 13** (*elpi_to_tree*). $\forall p \text{ v a r}, \text{runS } p \text{ v a r} \rightarrow$

$\forall \sigma t, \text{valid_tree } t \rightarrow \text{t2l } t \sigma \emptyset = a \rightarrow$
 if $r \text{ is } \cdot(\sigma', a')$ then
 $\exists t', \text{runT}' p \text{ v } \sigma t \cdot (\sigma', t') \wedge \text{t2l } (\text{odflt } KO t') \sigma \emptyset = a'$
 else $\text{runT}' p \text{ v } \sigma t \square$.

Proof. The proof is based on the idea explained in [2, Section 3.5.1] and depicted in the figure on the right: we have a state t whose translation is a , we proceed by induction on the execution of `runS`. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to the stack state is a' . This proof strategy is indirectly providing a kind of `12t` function by combining the execution of the `runS` and `t2l`.



Stating the converse direction in a symmetric way would have requires a function `12t` transforming an stack a to a tree t . Writing this function is far from trivial, since the structure of the tree is lost by `save_alts` and `run` that intuitively keep the state into a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover one would need to define a notion of valid stack that is equally intricate.

6 Case study: determinacy alanalysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the ELPI language [11]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate return at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking at compile time: if an inconsistency is detected, a fatal error explains why the current program may violate determinacy.

A function behaves deterministically if two main conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to a given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments, where outputs are generated wrt the the received inputs. A deterministic call relates to its inputs only one output. In the literature, we find several works about determinacy checking, namely [21, 16, 7]. These works needs a mode checker statically ensuring that in a query with ground inputs, each recursive call is performed with ground inputs.

In our mechanization, and in particular in the ELPI language, we adopt a special unification routine on input argument, called `matching` [1, 27]. Similarly to `unify`, `matching` takes the query-term t_1 and the pattern t_2 and returns an optional subsitution σ' . The query-term is the argument in input mode taken from a dynamic call and the pattern is the corresponding input argument in the chosen rule-head.

428 The function `matching` has the following property:

429 ► **Axiom 1** (`matching_disj`). $\forall t_1 t_2 \sigma \sigma', [disjoint_vars_tm\ t_1 \ \&\ \text{domf}\ \sigma] \rightarrow disjoint_tm\ t_1\ t_2 \rightarrow$
 $matching\ t_1\ t_2\ \sigma = \cdot\sigma' \rightarrow \exists e, \text{domf}\ \sigma' = \text{domf}\ \sigma \setminus e \wedge e \leq vars_tm\ t_2.$

430 If the two terms are disjoint and the variables in the support of σ are disjoint from the variables
 431 in t_1 , then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 \equiv \sigma' t_2$, and the
 432 variables in t_1 are not assigned in σ' .

433 The `bc` function uses `matching` or `unify` on the arguments q depending if the argument is in
 434 *input* or *output* mode. Below we axiomatize two important properties of our unifiers.

435 ► **Axiom 2** (`unif_trans`). $\forall t_1 t_2 t_3 \sigma, unify\ t_1\ t_2\ \sigma \rightarrow unify\ t_2\ t_3\ \sigma \rightarrow unify\ t_1\ t_3\ \sigma.$

436 ► **Axiom 3** (`match_unif`). $\forall t_1 t_2 \sigma, matching\ t_1\ t_2\ \sigma \rightarrow unify\ t_1\ t_2\ \sigma.$

437 In Figure 1, the rules of the program are equipped with three signatures, one per predicate.
 438 Besides the arity of each predicate, a signature specifies which arguments are inputs and which are
 439 outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are
 440 expected to be functions, as specified by the `func` keyword, whereas `r` is a relation, as indicated by
 441 the `pred` keyword. The `->` symbol separates inputs from outputs; therefore, all three predicates are
 442 expected to take an `int` as input and return an `int` as output.

443 The signatures of the program are stored in the `sig` projection of the program P passed to the
 444 interpreter and are encoded as described in [11]; that is, similarly to the code in Figure 1, we use
 445 arrows for term application, data types to encode base types such as `int`, and two special symbols to
 446 distinguish deterministic from non-deterministic predicates.

447 A program execution behaves deterministically if, given a program, a substitution, and a tree to
 448 the `runT` inductive, the execution either fails or, if it succeeds, the tree of alternatives is $\square$. Below,
 449 we define `is_det`.

450 ► **Definition 14** (`is_det (p σ v t)`). $\forall r, runT'\ p\ v\ \sigma\ t\ r \rightarrow r = \square \vee \exists \sigma', r = \cdot(\sigma', \square).$

451 The theorem we want to prove is the following.

452 ► **Theorem 15** (`det_check_call`). $\forall p\ \sigma\ t\ v,$
 $check_P\ p \rightarrow tm_is_det\ p.(sig)\ t \rightarrow is_det\ p\ \sigma\ v\ (Todo\ (call\ t)).$

453 Here, `checkP` denotes the function that checks the program with respect to mutual exclusion
 454 and rule checking, and `tm_is_det` denotes the function that tests whether the head of the term
 455 refers to a rigid constant with a deterministic type.

456 The proof of this lemma should proceed by induction on the derivation of the program execution.
 457 However, with the current definition, the induction hypothesis is too weak: it would be formulated
 458 over a `Todo` tree, whereas the conclusion may concern an `Or` tree, rendering the induction hypothesis
 459 unusable.

460 To address this issue, we introduce the notion of “deterministic trees” (`det_tree`), show that a
 461 tree satisfying this condition enjoys the desired properties, and then prove the following theorem,
 462 which is more general than `det_check_call`.

463 ► **Lemma 16** (`det_check_tree`). $\forall \sigma\ v\ p\ t, check_P\ p \rightarrow det_tree\ p.(sig)\ t \rightarrow is_det\ p\ \sigma\ v\ t.$

464 Since `det_check_call` is an instance of `det_check_tree`, its proof is now trivial.

465 As an important remark, proving the same lemma with the stack semantics, it is difficult to
 466 correctly define the predicate `det_stack`, checking for the deterministic behavior of a generic stack,
 467 since, similar to Section 3.5, the lack of structure make this goal hard. We need therefore an
 468 intermediate data structure, like the tree to encompass this problem.

7 Related work

We studied a PROLOG semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-Prolog’s “matching-clauses” [1, 27]. The two semantics we presented (and proved equivalent) also apply to standard PROLOG if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the development only as follows: Axiom 1 does not require the query q to be ground. Adapting our results to standard PROLOG therefore requires the insertion of two additional hypotheses in Theorem 15. First, we require that t has ground inputs. Second, we require that the program p is statically well-moded.

Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in Section 6 still apply.

The only mechanization of Prolog’s semantics we could find is the one by Pusch in ISABELLE/HOL [20] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [26, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in Section 4 that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing the stack based semantics equivalent to a more optimized one.

Another relevant work are the ones by King [15], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features our paper [11] does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2] and we thank Y.Bertot for insightful discussions on the topic. In [2] Bertot relates bla bla, without never constructing a decompiler, exactly as we never build a list to tree function.

FINIRE

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

- 1 Hassan Ait-Kaci. *Warren’s Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21,

2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34>, doi:10.4230/LIPIcs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:Debray1988.
- 7 Saumya K. Debray and David S. Warren. Functional computations in logic programs. volume 11, page 451–481. Association for Computing Machinery, July 1989. doi:10.1145/65979.65984.
- 8 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λ Prolog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.
- 9 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 10 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- 11 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- 12 Benjamin Grégoire, Jean-Christophe L  chenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
- 13 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.
- 14 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum f  r Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27>, doi:10.4230/LIPIcs.ITP.2025.27.
- 15 Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. volume 11, pages 537–553. Cambridge University Press, 2011.
- 16 Lunjin Lu and Andy King. Determinacy inference for logic programs. volume 3444, pages 108–123, 04 2005. doi:10.1007/978-3-540-31987-0_9.
- 17 math-comp. finmap — finite maps for mathcomp, 2026. URL: <https://github.com/math-comp/finmap>.
- 18 Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 19 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.
- 20 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
- 21 Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. volume 29, pages 17–64, 1996. High-Performance Implementations of Logic Programming Systems.

- 564 URL: <https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684>,
565 doi:10.1016/S0743-1066(96)00068-4.
- 566 **22** SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-
567 Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.
- 568 **23** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
569 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
570 2018. URL: <https://inria.hal.science/hal-01637063>.
- 571 **24** Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
572 *LIPICs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
573 doi:10.4230/LIPICs.CVIT.2016.23.
- 574 **25** Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
575 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 576 **26** David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
577 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
578 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
579 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).
- 580 **27** Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*,
581 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.